

Esame Programmazione di Sistema / API Programming

Data: 2025-01-13

QUESTIONS

Domanda 1

Dato il seguente programma si descriva il comportamento del programma e la mappa della memoria utilizzata dalle variabili:

```
1 fn main() {  
2     trait Calculate {  
3         fn add(&self, l: u64, r: u64) -> u64;  
4         fn mul(&self, l: u64, r: u64) -> u64;  
5     }  
6     struct Modulo(u64);  
7  
8     impl Calculate for Modulo {  
9         fn add(&self, l: u64, r: u64) -> u64 {  
10             (l + r) % self.0  
11         }  
12  
13         fn mul(&self, l: u64, r: u64) -> u64 {  
14             (l * r) % self.0  
15         }  
16     }  
17  
18     let mod3 = Modulo(3);  
19  
20     let tobj: &dyn Calculate = &mod3;  
21     let result = tobj.add(2, 2);  
22  
23     println!(" Result = {:?}", result);  
24 }
```

Domanda 2

Dato il seguente programma si motivi l'errore di compilazione e si corregga il programma in modo da stampare il valore aggiornato della variabile `count` dopo ogni incremento.

```
1 fn main() {  
2     let mut count = 0;  
3  
4     let mut increment_n = |n| {
```

```

5     count += n;
6 };
7
8     increment_n(10);
9     println!("{}", count);
10
11    increment_n(5);
12    println!("{}", count);
13 }

```

Domanda 3

Spiegare il comportamento del seguente programma.

```

1 fn call_x(mut f: impl FnMut(), x: i32) {
2     for _ in 0..x {
3         f();
4     }
5 }
6
7 fn main() {
8     let mut a = 0;
9     let count_a = move || { a += 1; print!("{}", a); };
10    a = 2;
11    call_x(count_a, 2);
12    call_x(count_a, 2);
13    println!("\t | a = {}", &a);
14    call_x(count_a, 4);
15    println!("\t | a = {}", &a);
16 }

```

Domanda 4

La struct `DelayedExecutor` permette di eseguire funzioni in modo asincrono, dopo un certo intervallo di tempo. Essa offre tre metodi:

- `new()` -> `Self`: crea un nuovo `DelayedExecutor`
- `execute<F: FnOnce()+Send+'static>(f: F, delay: Duration) -> bool`: se il `DelayedExecutor` è aperto, accoda la funzione `f` che dovrà essere eseguita non prima che sia trascorso un intervallo pari a `delay` e restituisce `true`; se invece il `DelayedExecutor` è chiuso, restituisce `false`.
- `close(drop_pending_tasks: bool)`: chiude il `DelayedExecutor`; se `drop_pending_tasks` è `true`, le funzioni in attesa di essere eseguite vengono eliminate, altrimenti vengono eseguite a tempo debito.

`DelayedExecutor` è thread-safe e può essere utilizzato da più thread contemporaneamente. I task sottomessi al `DelayedExecutor` devono essere eseguiti in ordine di scadenza. All'atto della distruzione di un `DelayedExecutor`, tutti i task in attesa sono eliminati, ma se è in corso un'esecuzione questa viene portata a termine evitando di creare corse critiche.

Si implementi questa struct in linguaggio Rust.

SOLUTIONS

Domanda 1

Il programma definisce una struttura `Modulo` che implementa il tratto `Calculate`. Viene quindi creata un'istanza `mod3` di questa struttura. Successivamente, si crea un riferimento d'oggetto tratto (trait object) `tobj` che punta a `mod3`. Infine, viene invocato il metodo `add` tramite il trait object e il risultato viene stampato.

Poiché la chiamata al metodo `add` viene effettuata tramite un trait object (`&dyn Calculate`), il compilatore utilizza il dynamic dispatch. `tobj` è un fat pointer composto da due elementi: un puntatore ai dati (all'istanza `mod3`) e un puntatore alla vtable associata al tipo `Modulo` per il tratto `Calculate`. Questo meccanismo introduce un piccolo overhead sia in termini di memoria (per via della vtable e del fat pointer) sia di prestazioni (per la dereferenziazione indiretta).

Mapa della memoria (assumendo un'architettura a 64 bit):

- **8 byte** per `mod3`: La struct `Modulo` contiene solo un campo `u64`.
- **16 byte** per `tobj`: Un fat pointer (`&dyn Calculate`) composto da un puntatore ai dati (8 byte) e un puntatore alla vtable (8 byte).
- **8 byte** per `result`: Il valore di ritorno del metodo `add` è un `u64`.

Tutte queste variabili (`mod3`, `tobj`, `result`) sono allocate sullo stack. Non ci sono allocazioni sull'heap in questo programma.

Domanda 2

L'errore di compilazione è dovuto alle regole di borrowing di Rust. La chiusura `increment_n` cattura un riferimento mutabile (`&mut count`) alla variabile `count`. Finché la chiusura è viva e in uso (che si estende fino all'ultima sua invocazione), il riferimento mutabile è attivo. Quando il `println!` tenta di accedere a `count` in sola lettura (`&count`), si verifica un conflitto perché non è possibile avere un borrow immutabile mentre è attivo un borrow mutabile.

Per correggere il problema in modo che venga stampato il valore aggiornato dopo ogni incremento, possiamo spostare la logica di stampa all'interno della chiusura stessa:

```
1 fn main() {  
2     let mut count = 0;  
3  
4     let mut increment_n = |n| {  
5         count += n;  
6         println!("{}", count);  
7     };  
8  
9     increment_n(10);  
10    increment_n(5);  
11 }
```

Domanda 3

Il tipo della variabile `a` è `i32`, che implementa il tratto `Copy`. A riga 8, la chiusura `count_a` viene dichiarata con la keyword `move`. Questo significa che la chiusura prende possesso delle variabili catturate. Tuttavia, poiché `i32` è `Copy`, la variabile `a` non viene realmente spostata (moved) bensì copiata all'interno della chiusura. Lo stato interno della chiusura contiene quindi una copia indipendente di `a`, inizializzata a 0.

Poiché lo stato catturato è composto unicamente da un tipo `Copy`, l'intera chiusura implementa automaticamente a sua volta il tratto `Copy`. Di conseguenza, ogni volta che `call_x` viene invocata (alle righe 10, 11 e 13), ad essa non viene passata la chiusura originale, ma una copia della chiusura `count_a` (ognuna con il suo stato iniziale indipendente `a = 0`). All'interno di `call_x`, la copia viene mutata ed esegue il suo ciclo, ma non affetta la chiusura originale nel `main` né l'ambiente esterno.

Nel `main`, a riga 9, la variabile `a` originale viene impostata a 2. Questo valore non verrà mai più alterato e rimane 2 per tutta l'esecuzione.

Output del programma:

- `call_x(count_a, 2)`: riceve una copia della chiusura con `a=0`. Stampa `1 2` .
- `call_x(count_a, 2)`: riceve una nuova copia con `a=0`. Stampa `1 2` .
- `println!`: stampa `| a = 2`
- `call_x(count_a, 4)`: riceve un'altra nuova copia con `a=0`. Stampa `1 2 3 4` .
- `println!`: stampa `| a = 2`

Output complessivo: `1 2 1 2 | a = 2`

`1 2 3 4 | a = 2`

Domanda 4

Ecco l'implementazione corretta in Rust della struct `DelayedExecutor`. Vengono utilizzati un `Arc<Mutex<...>>` per gestire lo stato condiviso e una `Condvar` per risvegliare il thread quando vengono inseriti nuovi task o in caso di chiusura. I task vengono ordinati in modo decrescente rispetto al tempo di esecuzione, così da poter facilmente estrarre (tramite `pop()`) quello con la scadenza più prossima che si troverà in fondo al vettore.

```
1 use std::sync::{Arc, Condvar, Mutex};
2 use std::thread::{self, JoinHandle};
3 use std::time::{Duration, Instant};
4
5 #[derive(Eq, PartialEq)]
6 enum ExecutorState {
7     Open,
8     Closed,
9 }
10
11 pub struct DelayedExecutor {
12     tasks: Arc<Mutex<(Vec<(Box<dyn FnOnce() + Send + 'static>, Instant)>, ExecutorState)>>,
13     cv: Arc<Condvar>,
14     jh: Option<JoinHandle<()>>,
15 }
```

```

16
17 impl Drop for DelayedExecutor {
18     fn drop(&mut self) {
19         self.close(true);
20         if let Some(handle) = self.jh.take() {
21             let _ = handle.join();
22         }
23     }
24 }
25
26 impl DelayedExecutor {
27     pub fn new() -> Self {
28         let tasks = Arc::new(Mutex::new((Vec::new(), ExecutorState::
29             Open)));
30         let cv = Arc::new(Condvar::new());
31
32         let tasks_c = Arc::clone(&tasks);
33         let cv_c = Arc::clone(&cv);
34
35         let jh = thread::spawn(move || {
36             let mut tasks_guard = tasks_c.lock().unwrap();
37
38             loop {
39                 while tasks_guard.0.is_empty() && tasks_guard.1 ==
40                     ExecutorState::Open {
41                     tasks_guard = cv_c.wait(tasks_guard).unwrap();
42                 }
43
44                 if !tasks_guard.0.is_empty() {
45                     let now = Instant::now();
46                     let last_idx = tasks_guard.0.len() - 1;
47                     let target_time = tasks_guard.0[last_idx].1;
48
49                     if target_time <= now {
50                         let (task, _) = tasks_guard.0.pop().unwrap();
51                         drop(tasks_guard);
52
53                         task(); // Esecuzione della funzione
54
55                         tasks_guard = tasks_c.lock().unwrap();
56                     } else {
57                         let wait_dur = target_time.duration_since(now);
58                         let (guard, _timeout_res) = cv_c.wait_timeout(
59                             tasks_guard, wait_dur).unwrap();
60                         tasks_guard = guard;
61                     }
62                 } else if tasks_guard.1 == ExecutorState::Closed {
63                     break;
64                 }
65             }
66         });
67
68         Self {
69             tasks,
70             cv,
71             jh: Some(jh),
72         }
73     }
74 }

```

```

71
72 pub fn execute<F>(&self, f: F, delay: Duration) -> bool
73 where
74     F: FnOnce() + Send + 'static,
75 {
76     let mut tasks_guard = self.tasks.lock().unwrap();
77     if tasks_guard.1 == ExecutorState::Closed {
78         return false;
79     }
80
81     let exec_time = Instant::now() + delay;
82     tasks_guard.0.push((Box::new(f), exec_time));
83     tasks_guard.0.sort_by(|a, b| b.1.cmp(&a.1)); // decrescente
84
85     self.cv.notify_one();
86     true
87 }
88
89 pub fn close(&self, drop_pending_tasks: bool) {
90     let mut tasks_guard = self.tasks.lock().unwrap();
91     tasks_guard.1 = ExecutorState::Closed;
92
93     if drop_pending_tasks {
94         tasks_guard.0.clear();
95     }
96
97     self.cv.notify_one();
98 }
99 }

```